

LAB 01 | المختبر 01

Neural Network Training

تدريب شبكة عصبية من الحل الابتدائي حتى التقارب

Forward Propagation, Backpropagation, Weight Update, and Convergence

الانتشار الأمامي والانتشار الخلفي وتحديث الأوزان والوصول إلى التقارب

Prepared by | إعداد: Dr.-Ing. Aouss Gabash



Learning Outcomes

مخرجات التعلم

1. Implement a MATLAB workflow for artificial intelligence foundations.

تنفيذ سير عمل في MATLAB حول أسس الذكاء الاصطناعي.

2. Interpret numerical results, plots, and engineering implications.

تفسير النتائج العددية والرسوم والدلالات الهندسية.

3. Validate the implementation and discuss limitations.

التحقق من صحة التنفيذ ومناقشة حدوده.



Prerequisite sites

المتطلبات

السابقة

Basic mathematics and introductory electrical engineering.

الرياضيات الأساسية

ومبادئ الهندسة الكهربائية.

Artificial Intelligence

Foundations | أسس

الذكاء الاصطناعي



Lab Objectives

- Implement a small neural network in MATLAB.
- Calculate forward propagation and loss.
- Implement analytical backpropagation.
- Update weights using gradient descent.
- Stop training when the error becomes smaller than ϵ .

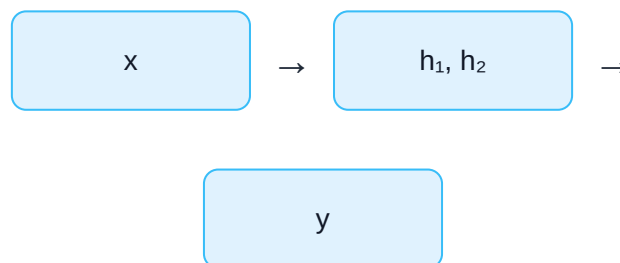
أهداف المختبر

- تنفيذ شبكة عصبية صغيرة باستخدام MATLAB.

- حساب الانتشار الأمامي ودالة الخطأ.
- تنفيذ الانتشار الخلفي تحليليًا.
- تحديث الأوزان باستخدام الانحدار المتدرج.
- إيقاف التدريب عندما يصبح الخطأ أصغر من ϵ .

1. Network Structure

1 Input $\rightarrow$ 2 Hidden Neurons $\rightarrow$ 1 Output



1. بنية الشبكة

1 $\rightarrow$ 2 $\rightarrow$ 1

دخل واحد، عصبونان مخفيان، وخرج واحد.

2. Activation Function

$$\sigma(z) = 1/(1 + e^{-z})$$

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$$

2. دالة التنشيط

$$\sigma(z) = 1/(1 + e^{-z})$$

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$$

3. MATLAB Sigmoid Function

```
sigma = @(z) 1 ./ (1 + exp(-z));
```

3. دالة Sigmoid في MATLAB

يحسب هذا السطر قيمة Sigmoid لأي قيمة z .

4. Initial Parameters

```
W.w1 = 0.4; W.w2 = -0.3;  
W.v1 = 0.2; W.v2 = 0.5;  
W.b1 = 0.1; W.b2 = -0.1; W.b3 = 0.0;
```

These are initial values; training will modify them.

4. القيم الابتدائية

تبدأ الشبكة بأوزان وانزياحات أولية ثم تعدلها أثناء التدريب.

5. Training Sample

```
x = 1;  
t = 0.8;
```

$x = input, t = desiredtarget$

5. عينة التدريب

x هو الدخل و t هي القيمة المطلوبة.

6. Forward Propagation

$$z_1 = w_1x + b_1, h_1 = \sigma(z_1)$$

$$z_2 = w_2x + b_2, h_2 = \sigma(z_2)$$

$$z_3 = v_1h_1 + v_2h_2 + b_3, y = \sigma(z_3)$$

6. الانتشار الأمامي

يتم حساب خرج العصبونين المخفيين ثم استخدامهما لحساب خرج الشبكة النهائي.

7. MATLAB Forward Propagation

```
z1 = W.w1*x + W.b1; h1 = sigma(z1);  
z2 = W.w2*x + W.b2; h2 = sigma(z2);  
z3 = W.v1*h1 + W.v2*h2 + W.b3;  
y = sigma(z3);
```

7. الانتشار الأمامي في MATLAB

تنفذ هذه الأسطر المعادلات السابقة مباشرة.

8. Loss Function

$$E = 1/2(y - t)^2$$

$$E = 0.5*(y - t)^2;$$

8. دالة الخطأ

كلما اقترب y من t انخفضت E .

9. Output Error

$$\delta_3 = (y - t)y(1 - y)$$

$$d0 = (y - t) * y * (1 - y);$$

$d0$ represents $\partial E / \partial z_3$.

9. خطأ طبقة الخرج

تمثل d0 تأثير دخل عصبون الخرج على الخطأ النهائي.

10. Output-Layer Gradients

$$\partial E / \partial v_1 = \delta_3 h_1, \partial E / \partial v_2 = \delta_3 h_2, \partial E / \partial b_3 = \delta_3$$

$$dV1 = d0 * h1; \quad dV2 = d0 * h2; \quad dB3 = d0;$$

10. تدرجات طبقة الخرج

نحسب تأثير كل وزن وانزياح في طبقة الخرج على دالة الخطأ.

11. Hidden-Layer Error

$$\delta_1 = \delta_3 v_1 h_1 (1 - h_1), \delta_2 = \delta_3 v_2 h_2 (1 - h_2)$$

$$d1 = d0 * W.v1 * h1 * (1 - h1);$$

$$d2 = d0 * W.v2 * h2 * (1 - h2);$$

11. خطأ الطبقة المخفية

ينتقل تأثير الخطأ من الخرج إلى العصبونات المخفية عبر الأوزان V_1 و V_2 .

12. Input-to-Hidden Gradients

$$\partial E / \partial w_1 = \delta_1 x, \partial E / \partial w_2 = \delta_2 x$$

$$\begin{aligned} dW1 &= d1 * x; & dB1 &= d1; \\ dW2 &= d2 * x; & dB2 &= d2; \end{aligned}$$

12. تدرجات الأوزان الأولى

نحصل على مشتقات الخطأ بالنسبة للأوزان والانزياحات بين الدخل والطبقة المخفية.

13. Gradient Descent Update

$$w(new) = w(old) - \eta \partial E / \partial w$$

$$\begin{aligned} W.w1 &= W.w1 - \eta * dW1; & W.w2 &= W.w2 - \eta * dW2; \\ W.v1 &= W.v1 - \eta * dV1; & W.v2 &= W.v2 - \eta * dV2; \\ W.b1 &= W.b1 - \eta * dB1; & W.b2 &= W.b2 - \eta * dB2; \\ W.b3 &= W.b3 - \eta * dB3; \end{aligned}$$

13. تحديث الأوزان

يتم تحديث كل معامل بعكس اتجاه التدرج لتقليل الخطأ.

14. Learning Rate

```
eta = 0.5;
```

η too small

Training becomes slow.

η too large

Training may oscillate or diverge.

14. معدل التعلم

القيمة الصغيرة جدًا تبطئ التدريب، والكبيرة جدًا قد تجعل التدريب غير مستقر.

15. Stopping Criterion

```
epsilon = 1e-6;  
maxEpochs = 10000;
```

Stop when $E < \epsilon$

15. معيار التوقف

يتوقف التدريب عندما يصبح الخطأ أصغر من ٤ أو يصل إلى الحد الأقصى للدورات.

16. Complete MATLAB Program

```

clear; clc; close all;
sigma = @(z) 1 ./ (1 + exp(-z));
x = 1; t = 0.8;
eta = 0.5; epsilon = 1e-6; maxEpochs = 10000;
W.w1 = 0.4; W.w2 = -0.3;
W.v1 = 0.2; W.v2 = 0.5;
W.b1 = 0.1; W.b2 = -0.1; W.b3 = 0.0;
lossHist = zeros(1,maxEpochs);
epoch = 0; converged = false;

while ~converged && epoch < maxEpochs
    epoch = epoch + 1;
    z1 = W.w1*x + W.b1; h1 = sigma(z1);
    z2 = W.w2*x + W.b2; h2 = sigma(z2);
    z3 = W.v1*h1 + W.v2*h2 + W.b3; y = sigma(z3);
    E = 0.5*(y - t)^2; lossHist(epoch) = E;
    d0 = (y - t) * y * (1 - y);
    dV1 = d0*h1; dV2 = d0*h2; dB3 = d0;
    d1 = d0*W.v1*h1*(1-h1);
    d2 = d0*W.v2*h2*(1-h2);
    dW1 = d1*x; dB1 = d1;
    dW2 = d2*x; dB2 = d2;
    W.w1 = W.w1 - eta*dW1; W.w2 = W.w2 - eta*dW2;
    W.v1 = W.v1 - eta*dV1; W.v2 = W.v2 - eta*dV2;
    W.b1 = W.b1 - eta*dB1; W.b2 = W.b2 - eta*dB2;
    W.b3 = W.b3 - eta*dB3;
    converged = E < epsilon;
end

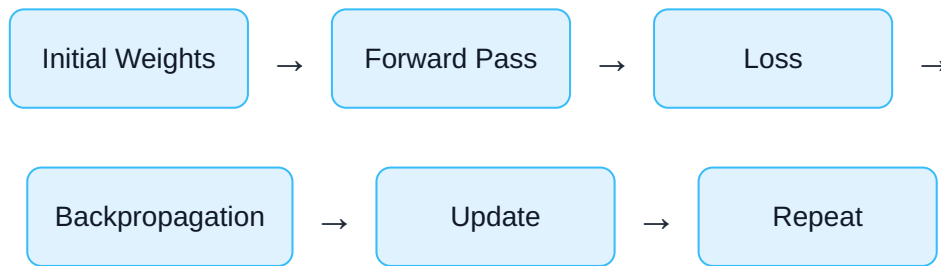
disp(W)
fprintf('Epochs = %d\n',epoch)
fprintf('Final output y = %.6f\n',y)
fprintf('Target t = %.6f\n',t)
fprintf('Final error E = %.10f\n',E)
figure
semilogy(1:epoch,lossHist(1:epoch),'LineWidth',1.6)
grid on
xlabel('Epoch'); ylabel('Loss E')
title('Neural Network Training Convergence')

```

16. البرنامج الكامل في MATLAB

يجمع البرنامج الانتشار الأمامي وحساب الخطأ والانتشار الخلفي وتحديث الأوزان وفحص التقارب ضمن حلقة تدريب واحدة.

17. Training Loop



17. حلقة التدريب

الأوزان الابتدائية ← الانتشار الأمامي ← الخطأ ← الانتشار الخلفي ← التحديث ← التكرار

18. Convergence Plot

```
semilogy(1:epoch,lossHist(1:epoch),'LineWidth',1.6)
```

A descending curve indicates that the network is learning.

18. منحنى التقارب

يظهر المقياس اللوغاريتمي انخفاض الخطأ عبر عدة مراتب عشرية بوضوح.

19. Experiment 1 — Learning Rate

```
eta = 0.05;  
eta = 0.5;  
eta = 2.0;
```

Compare epochs, stability, and final loss.

19. التجربة 1 — معدل التعلم

غير η وقارن سرعة التقارب والاستقرار والخطأ النهائي.

20. Experiment 2 — Initial Weights

```
W.w1  
W.w2  
W.v1  
W.v2
```

Different initial conditions may produce different training paths.

20. التجربة 2 — الأوزان الابتدائية

غير القيم الابتدائية ولاحظ تأثيرها على مسار التدريب وعدد الدورات.

21. Power-System Interpretation

$$x = PreviousLoad, t = FutureLoad$$

$$x = [Load, Temperature, Time, PVPower, ...]$$

The mathematics of backpropagation remains the same.

21. التفسير في نظم الطاقة

يمكن استخدام آلية التدريب نفسها للتنبؤ بالحمل أو الطاقة المتجددة أو تقدير حالات التشغيل.

22. Lab Tasks

1. Run the original program.
2. Record final output and epochs.
3. Change the learning rate.
4. Change the initial weights.
5. Change the target t.
6. Explain every gradient expression.
7. Explain why the loss decreases.

22. مهام المختبر

1. شغل البرنامج.

2. سجل الخرج وعدد الدورات.

3. غير معدل التعلم.
4. غير الأوزان الابتدائية.
5. غير t .
6. اشرح معادلات التدرج.
7. اشرح سبب انخفاض الخطأ.



Questions

1. What does dO represent?
2. Why does dO contain $y(1-y)$?
3. Why is $dV1=dO*h1$?
4. Why does the hidden error contain $W.v1$?
5. Why do we subtract the gradient?
6. What happens if η is too large?
7. What is the role of ϵ ?

أسئلة المختبر

1. ماذا تمثل dO ؟
2. لماذا تحتوي على $y(1-y)$ ؟
3. لماذا $dV1=dO*h1$ ؟
4. لماذا يحتوي خطأ الطبقة المخفية على $W.v1$ ؟
5. لماذا نطرح التدرج؟
6. ماذا يحدث إذا كانت η كبيرة؟
7. ما وظيفة ϵ ؟

References | المراجع

المراجع العامة المستخدمة في هذا المقرر

- [1] A. Gabash, *Flexible Optimal Operations of Energy Supply Networks with Renewable Energy Generation and Battery Storage*. Saarbrücken, Germany: Südwestdeutscher Verlag für Hochschulschriften, 2014.
- [2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th Global ed. Harlow, U.K.: Pearson Education Limited, 2022.
- [3] A. Gabash, "Energy Market Transition and Climate Change: A Review of TSOs-DSOs C+++ Framework from 1800 to Present," *Energies*, vol. 16, no. 17, Art. no. 6139, 2023, doi: 10.3390/en16176139.
- [4] A. Gabash, *AI Applications in Electrical Power Systems*, Version 1.0, 2026. [Online]. Available: <https://aoussgabash.com>

Recommended citation:

Gabash, A. (2026). Neural Network Training. AI Applications in Electrical Power Systems, Laboratory 01, Version 1.0. <https://aoussgabash.com>

المرجع المقترح:

أوس غباش، 2026، مخبر 01، تطبيقات الذكاء الاصطناعي في أنظمة الطاقة الكهربائية، الإصدار 1.0، <https://aoussgabash.com>

© 2026 Dr.-Ing. Aouss Gabash. Educational use with attribution to the author and official website.